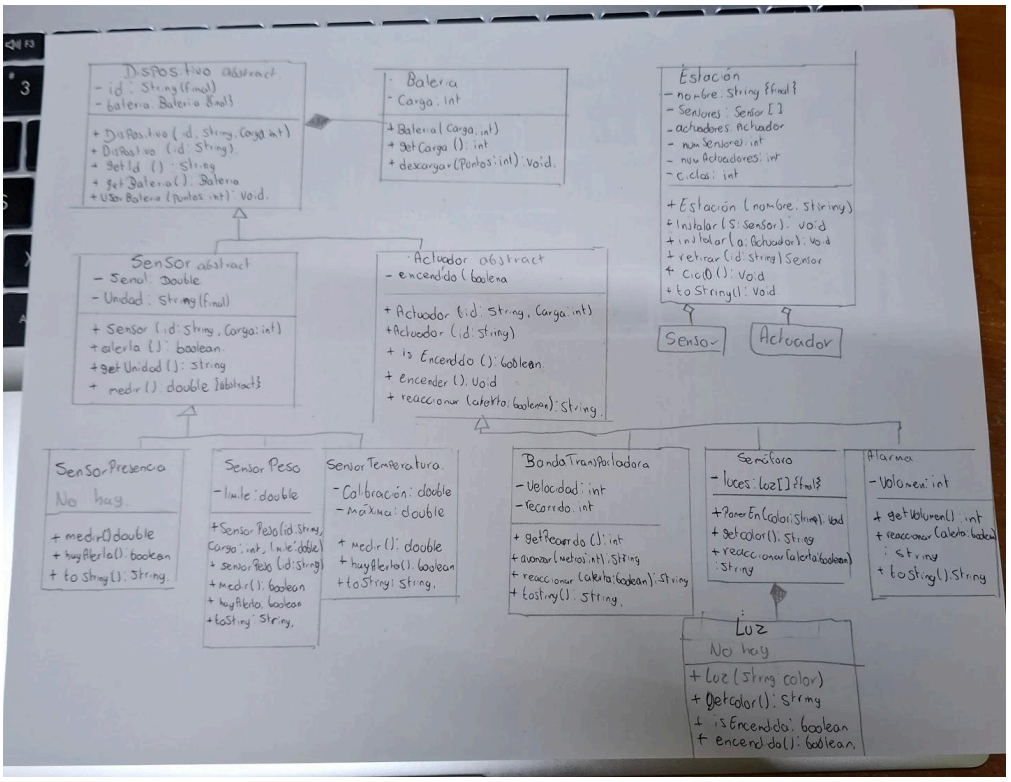
Flores Diaz Diego Adair 323294506
Abraham Lopez




```
diego@LAPTOP-DR7RBISN MINGW64 ~/mpoo/mpoo-2
$ java estacion.PruebaTemperatura
Sensor S3 (100%) temperatura, max 45.0
S3 mide 40.0 grados
S3 mide 48.5 grados ALERTA
Sensor S3 (98%) temperatura, max 45.0
```

```
$ java estacion.Main
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%)  peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
S1 mide 12.0 cm
S2 mide 18.5 kg
S3 mide 40.0 grados
A1 avanza 2 m, lleva 2 m
A2 en verde
A3 en silencio
== Ciclo 2 en Anden 3
S1 mide 9.5 cm
S2 mide 32.0 kg ALERTA
S3 mide 42.0 grados
A1 se detiene
A2 en rojo
A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
S1 mide 15.0 cm
S3 mide 46.5 grados ALERTA
A1 se detiene
A2 en rojo
A3 suena a volumen 10
Regresa: Sensor S2 (78%)  peso, limite 25.0
Instalado: Sensor S2 (78%)  peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
```

Flores Diaz Diego Adair 323294506
Abraham Lopez

1. **Sobrecarga contra sobrescritura:** un ejemplo de cada una sacado de tu código, y la diferencia.

sobrecarga:

```
public Sensor(String id, String unidad, int carga) {
      super(id, carga);
      this.unidad = unidad;
   }

   public Sensor(String id, String unidad) {
      this(id, unidad, 100);
   }
```

sobreescritura:

```
@Override
public String toString() {
   return "Sensor "  + super.toString();
}
```

La diferencia es que la sobrecarga modifica el comportamiento en su misma clase, y la sobreescritura modifica el comportamiento de una clase heredada del padre.

2. **Composición contra agregación:** ¿por qué la Batería es composición y el sensor en la Estación es agregación? Piensa en quién hace el new y en qué le pasó a S2 en el ciclo 3.

La batería es composición porque cada dispositivo crea su propia batería con new, forma parte de él. La estación tiene agregación con los sensores porque no los crea, sino que los recibe de main. S2 en el ciclo 3 se retira de la estación para mantenimiento y después regresa siendo el mismo objeto, conservando su bateria.

3. **Modificadores**: agrega en Main la línea bascula.gastar (5);, copia el error y explica por qué SensorPeso sí puede usar gastar y Main no. Bórrala antes de entregar.

```
javac estacion/main.java
estacion\main.java:53: error: cannot find symbol
      bascula.gastar(5);
           ^
  symbol:   method gastar(int)
  location: variable bascula of type SensorPeso
1 error
```

SensorPeso si puede usar gastar porque es un método private dentro de la misma clase, por lo tanto tiene total acceso a sus métodos, mientras que main no tiene acceso a los métodos privados de sensorpeso por eso no la reconoce.